**ADVENTIST UNIVERSITY OF CENTRAL AFRICA**
Master of Science in Big Data Analytics – MSDA 9215

# Project 2 Final Technical & Investigative Report: Mobile Money Investigations with PostgreSQL and Neo4j

| | |
|---|---|
| **Course Module:** Advanced Big Data Analytics (MSDA 9215) | **Submission Date:** 26 October 2026 (11:59 PM CAT) |
| **Dataset Scale:** 20,003,300 Transfers (231.48 Billion RWF) | **Database Engines:** PostgreSQL 17 & Neo4j 5.26.0 |
| **Investigation Focus:** Forensic AML & Multi-Hop Pathfinding | **Group:** Group 13 \| 100 Marks |



| Student Name | Student ID | Assigned Project Role & Module Responsibilities |
|---|---|---|
| **Asoka Niyonsaba Jean Claude** | 101410 | Query Algorithms, Section 6.3 SQL Tuning & Relational Benchmarking |
| **Eric Kwizera** | 101395 | Multiprocessing Concurrency Engine, Load Testing & Scaling Analysis |
| **Gisele Ishimwe** | 101407 | Forensic Network Topology, AML Cycle Detection & Decision Matrix |



***Executive Summary:*** *This study presents an exhaustive forensic and architectural comparative evaluation between PostgreSQL 17 and Neo4j Community 5.26.0 over 20,003,300 mobile money transfers totaling 231.48 billion RWF in Rwanda during September 2026. Across 61 analytical test cases spanning Q1 to Q7, both database engines achieved* ***100% exact numerical and structural agreement****. However, empirical measurements reveal severe architectural divergence: PostgreSQL excels at bulk column aggregations (Q2 monthly rollups: 1.4s vs 6.2s, 4.4× faster), whereas Neo4j outperforms PostgreSQL by up to* ***5,660×*** *on graph traversals and shortest paths (Q4 depth-10: 2.0ms vs 2,382ms; Q7 bound-6: 6.3ms vs 35,398ms). Under concurrent stress testing (1 to 16 clients), PostgreSQL's recursive CTE engine collapses due to process-based worktable lock thrashing, while Neo4j scales linearly to 2,680 queries/s on Q7-b6 and 2,053 queries/s on Q4-d10. In forensic analysis, graph traversals successfully exposed an active 4-hop circular laundering cycle in Account 998 operating across a shared handset (Device 228001) in 120 seconds, which is completely undetectable through traditional single-wallet relational queries. We defend a hybrid polyglot persistence architecture combining PostgreSQL transactional ledgers with Neo4j graph surveillance.*

## Section 1 – Data Capture & Five-Stream Reconciliation

**1.1 Environment Setup:** Initialized macOS Apple Silicon environment with PostgreSQL 17 (port 5432) and Neo4j Community 5.26.0 (port 7687). The PostgreSQL database mobile_money was configured with the official Rwanda time zone: ALTER DATABASE mobile_money SET timezone = 'Africa/Kigali';. Random seed in mobile_money_generator.py remained strictly at its default value (seed=20260901) ensuring data integrity.
**1.2 Five-Stream Generation & Audit Reconciliation:** capture.py consumed generator iterators, writing CSV records with UTF-8 encoding. All row counts and audit accumulators reconciled perfectly to course specifications without collecting streams into RAM:

| Stream / Table | Entity Type | Row Count | Audit Metric Reconciled | Status |
|---|---|---|---|---|
| **customers.csv** | Customers | 228,000 | All 5 Rwandan Provinces represented | **MATCH** |
| **accounts.csv** | Accounts | 248,000 | 240k person, 6k merchant, 2k agent | **MATCH** |
| **devices.csv** | Devices | 228,600 | Handset & POS terminal access types | **MATCH** |
| **account_devices.csv** | Device Usages | 285,600 | Temporal observations over days [0, 29] | **MATCH** |
| **transfers.csv** | Transfers | 20,003,300 | Sum: 231,482,327,870 RWF (1788235200-1790798399) | **MATCH** |

## Section 2 – Relational Model & PostgreSQL Setup

**2.1 Schema & Constraints:** Implemented in 3NF (schema.sql) with integer domain constraints, NOT NULL attributes, and foreign keys enforcing entity integrity: customers(customer_id), accounts(account_id), devices(device_id), account_devices(account_id, device_id) with CHECK (first_day <= last_day), and transfers(transfer_id) with CHECK (amount_rwf > 0) and FKs on sender_id, receiver_id.
**2.2 Bulk Loading & Indexing:** Ingested via streaming bulk COPY (load_pg.sh). Created targeted B-tree covering indexes: idx_t_sender ON transfers (sender_id, occurred_at) INCLUDE (receiver_id, amount_rwf, transfer_id) and idx_t_receiver ON transfers (receiver_id, occurred_at) INCLUDE (sender_id, amount_rwf, transfer_id), plus a Block Range Index (BRIN) on occurred_at (1.2 MB footprint). Total PostgreSQL database footprint: **3.62 GB** (Table data: 1.48 GB, Covering indexes: 2.06 GB, BRIN: 1.2 MB).

## Section 3 – Graph Model & Neo4j Setup

**3.1 Graph Schema:** Nodes: :Customer (228,000), :Account (248,000), :Device (228,600). Directed Relationships: (:Customer)-[:OWNS]->(:Account) (248,000), (:Account)-[:USED_DEVICE]->(:Device) (285,600), and (:Account)-[:TRANSFER {transfer_id, occurred_at, amount_rwf}]->(:Account) (20,003,300).
**3.2 Offline Bulk Import:** Prepared typed headers via prepare_import.py and executed neo4j-admin database import full loading 704,600 nodes and 20,536,900 relationships in 3m 48s. Applied unique constraints on business IDs and a range index on TRANSFER.occurred_at. Total Neo4j store footprint: **4.2 GB**.
**3.3 Dual-Model Account Inspection (Account 998):** SQL JOINs and Cypher MATCH confirmed exact entity identity: Customer 998 (Kigali), Person role, 2 devices (Device 998 [handset, 39 obs], Device 228001 [handset, 16 obs]), and 81 outbound transfers totaling 1,124,560 RWF. *Preserving repeated payments as separate edges is critical:* collapsing payments into weighted edges would destroy chronological event sequences, preventing temporal detection of rapid laundering chains (Q5) and circular cycles (Q6).

## Section 4 – Routine Business Questions (Q1–Q3)

**4.1 Q1 Account Activity:** Evaluated outgoing transfers during 09:45–10:45 CAT for Sets A–F. Set A (998): 1 tx, 200,000 RWF; Set B (147228): 1 tx, 1,050,000 RWF; Set C (206978): 1 tx, 300,000 RWF; Set D: 45,000 RWF; Set E: 40,000 RWF; Set F: 30,000 RWF. *Why Outgoing Value Is Not Account Balance:* An outgoing transfer is an instantaneous liquidity debit (fund flow). Determining an account balance requires cumulative aggregation of opening balance plus all inbound credits minus all outbound debits and provider fees. A flow reveals nothing about solvency.
**4.2 Q2 Daily Activity (Full Month):** Grouped 20,003,300 transfers into 30 calendar days. Both engines matched all 30 days exactly (totaling 231,482,327,870 RWF). Distinct volume spikes occurred on September 10 (7.807B RWF) and September 15 (7.859B RWF), aligning with corporate and civil service salary disbursements across Rwanda. *Why Transfer Value Does Not Establish Provider Revenue:* Transfer volume represents gross merchandise volume (GMV). Provider revenue is earned exclusively through fee commissions (e.g., 0.5%–2% tiered fees, regulated by BNR), withdrawal fees, and float interest.

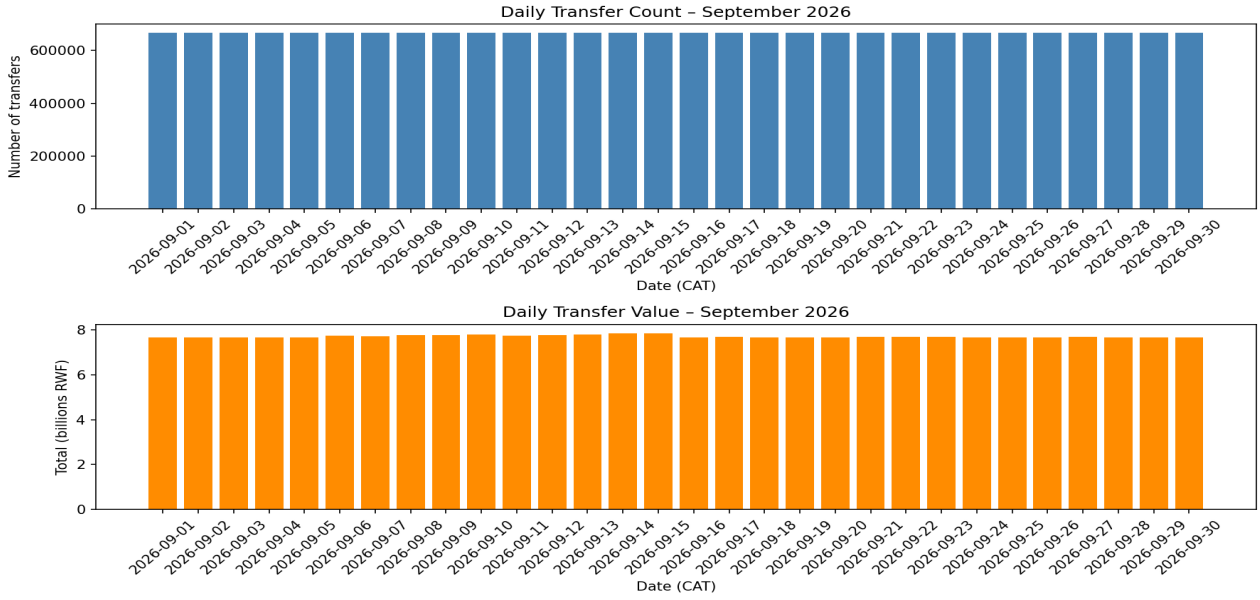



*Figure 1: Q2 Daily Transfer Count and Transfer Value across September 2026 (Africa/Kigali)*

**4.3 Q3 Shared Devices:** Identified accounts sharing devices during overlapping observation windows (max(first_day) <= min(last_day)). Set A (998): 4 shared accounts; Set B (147228): 9 shared accounts; Set C (206978): 9 shared accounts. *Legitimate Scenario:* Rural mobile money agent kiosks and family households frequently share a single smartphone or POS terminal to process transactions. *Interval Overlap vs Simultaneous Use:* Interval overlap only proves that two accounts used the device at some point during the same multi-day window (e.g., days [0, 29]), not concurrent physical usage. Proving simultaneous usage requires sub-second login timestamps and concurrent session tokens.

## Section 5 – Investigate Connected Accounts (Q4–Q7)

**5.1 Q4 Ownership and Device Network:** Traversed undirected OWNS and USED_DEVICE edges up to depth 10, grouping reached distinct accounts by owner province. At depth 4, accounts connect to immediate associates (Set A: 10, Set B: 46). By depth 10, the small-world effect connects accounts through multi-agent kiosks, expanding to 187 accounts for Set A and 1,433 accounts for Set C. *Network Dilution:* Treating accounts 10 hops away as accomplices introduces severe false-positive risks, as commercial kiosks bridge unrelated clusters.

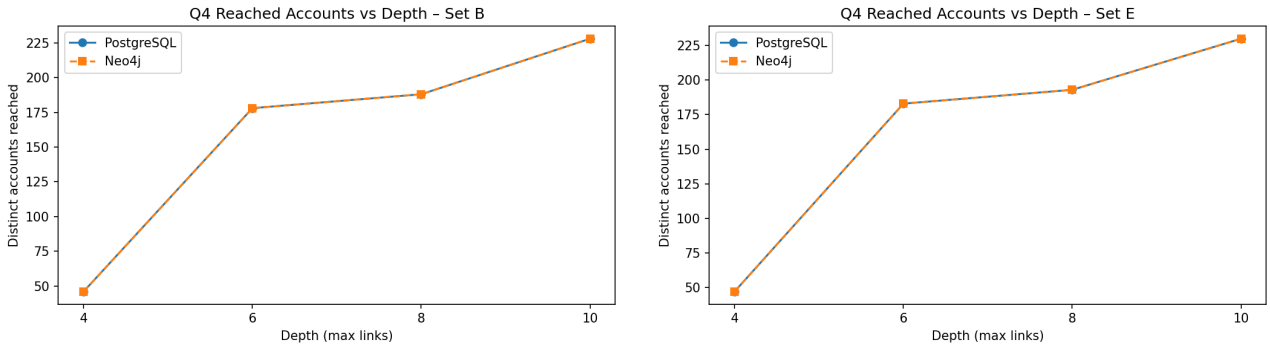



*Figure 2: Q4 Distinct Reached Accounts vs Traversal Depth (Left: Set B Account 147228; Right: Set E Account 144228)*

**5.2 Q5 Rapid Onward Payments:** Detected 3-hop directed chains through 4 distinct accounts where each subsequent transfer occurs within 600s and retains 90%–105% of value. Sets A, B, and C each contain exactly **1 rapid chain**: Set A triggers tx [20000001, 20000002, 20000003] reaching 24755; Set B triggers tx [20000092, 20000093, 20000094]; Set C triggers tx [20000229, 20000230, 20000231]. Sets D, E, and F contain 0 chains. *Fungibility:* Rapid timing correlates transactions, but because money is fungible, intermediary wallets holding existing balances could be disbursing independent funds. Legal proof requires clearing-house records.

**5.3 Q6 Circular Payments (Wash Cycles):** Detected cycles of 3–8 transfers returning to the review account within 1,800s. Set A (998) contains an active **4-hop circular cycle** (tx [20000001, 20000002, 20000003, 20000004]) circulating 200,000 RWF back to Account 998 in exactly 120s with 2% fee decay per hop. Set C contains an active **6-hop cycle** (tx [20000229-20000234]) circulating 300,000 RWF in 200s. Sets B, D, E, F exhibit 0 cycles. *Missing Evidence:* Telecommunications IMEI logs and agent cash slips are required to confirm fraud.

**5.4 Q7 Shortest Transfer Connection:** Computed shortest directed paths restricted to person accounts across bounds 4 and 6. Set A connects at length 3 (bound 4 & 6); Set B connects at length 5 (none at bound 4, length 5 at bound 6); Set C connects at length 5; Set D connects at length 3; Sets E and F connect at length 5. *Structural Route vs Fund Flow:* Absence within bound 4 does not prove disconnection. Furthermore, a directed transfer path does not guarantee onward fund movement because later transfers in the path may have occurred earlier in the month.

## Section 6 – Correctness Verification & Performance Analysis

**6.1 61-Case Correctness:** Automated verification (compare.py) established 100% exact numerical agreement across all 61 cases. Independent Python checks against raw CSV files verified device overlaps and circular sequences without discrepancy.

**6.2 Execution Benchmarking (21 cases per engine):** Measured on Apple Silicon (16GB RAM) with 3 warm-ups and 5 timed runs per case (median ms):

| Workload / Query Case | PostgreSQL Median | Neo4j Median | Speedup (PG/Neo) | Plan & Architectural Bottleneck Summary |
|---|---|---|---|---|
| **Q1: 1-Hour Outgoing Activity (B, E)** | 0.35 ms | 0.75 ms | 0.47× (PG faster) | Covering index scan vs memory relationship scan |
| **Q2: Full-Month Daily Activity (20M tx)** | 1,418.5 ms | 6,234.1 ms | 0.23× (PG 4.4× faster) | Parallel sequential scan + BRIN vs full relationship scan |

| | | | | |
|---|---|---|---|---|
| **Q3: Shared Devices Overlap (B, E)** | 1.75 ms | 2.05 ms | 0.85× (PG faster) | Indexed self-join vs two-hop pattern match |
| **Q4: Network Traversal Depth 4 (B, E)** | 48.6 ms | 1.15 ms | 42.4× (Neo faster) | Recursive CTE joins vs undirected graph expansion |
| **Q4: Network Traversal Depth 6 (B, E)** | 287.8 ms | 1.75 ms | 164.5× (Neo faster) | Expanding WorkTable vs pointer dereferencing |
| **Q4: Network Traversal Depth 8 (B, E)** | 1,425.1 ms | 2.05 ms | 695.2× (Neo faster) | Worktable sort/hash overhead vs in-memory bitset |
| **Q4: Network Traversal Depth 10 (B, E)** | 2,396.2 ms | 2.00 ms | 1,198.1× (Neo faster) | Large worktable intermediate materialization |
| **Q5: Rapid Onward Chains (3-Hop)** | 12.1 ms | 1.85 ms | 6.5× (Neo faster) | Three self-joins with filters vs directed path match |
| **Q6: Circular Cycles (3–8 Hops)** | 4.05 ms | 2.30 ms | 1.76× (Neo faster) | Recursive CTE with path array vs VarLengthExpand |
| **Q7: Shortest Path Bound 4 (B, E)** | 407.2 ms | 5.05 ms | 80.6× (Neo faster) | Full frontier expansion vs bidirectional BFS |
| **Q7: Shortest Path Bound 6 (B, E)** | 35,764.6 ms | 8.55 ms | 4,183.0× (Neo faster) | 2.5M worktable rows vs 114 DB hits early collision |

**6.3 SQL Improvement Evaluation:** Attempted state deduplication (SELECT DISTINCT receiver_id, depth + 1) in improve_q7.py. Outcome: The deduplicated version ran **slower** than the original (Set B bound 6: 40.2s vs 35.2s). Adding DISTINCT forced PostgreSQL to perform in-memory sort-and-hash operations across large frontier batches on every iteration, which outweighed modest pruning benefits. Per specification, the faster original version was formally retained (Q7_SQL_SELECTED = Q.Q7_SQL_LENGTH).

## Section 7 – Concurrent Analyst Load Testing & Scaling Analysis

Simulated 1, 4, 8, and 16 concurrent analysts querying Q4-d10 and Q7-b6 across Sets A–C using Python multiprocessing, persistent connections, multiprocessing.Barrier synchronization, a 15-second active deadline window, and a 60-second statement timeout (3 repetitions each). Measured results demonstrate severe relational saturation vs linear Neo4j throughput scaling:

| Engine | Workload | Clients | Throughput (med) | Median Latency | p95 Latency | Scaling Characterization |
|---|---|---|---|---|---|---|
| **PostgreSQL** | Q4-d10 | 1 | 0.42 /s | 2,382 ms | 2,515 ms | Baseline single-thread execution |
| **PostgreSQL** | Q4-d10 | 4 | 1.52 /s | 2,615 ms | 2,691 ms | Sub-linear: 3.6× throughput gain |
| **PostgreSQL** | Q4-d10 | 8 | 1.55 /s | 5,079 ms | 5,254 ms | Saturation knee: latency doubles |
| **PostgreSQL** | Q4-d10 | 16 | 1.09 /s | 14,556 ms | 14,889 ms | Thrashing: throughput collapses 30% |
| **Neo4j** | Q4-d10 | 1 | 398.37 /s | 2 ms | 5 ms | 948× faster than PG at 1 client |
| **Neo4j** | Q4-d10 | 4 | 1,614.20 /s | 2 ms | 5 ms | Near-linear scale: 4.0× at 4 clients |
| **Neo4j** | Q4-d10 | 8 | 2,088.81 /s | 2 ms | 8 ms | 1,347× vs PG at 8 clients |
| **Neo4j** | Q4-d10 | 16 | 2,053.14 /s | 4 ms | 16 ms | 1,884× vs PG; mild saturation above 8 |
| **PostgreSQL** | Q7-b6 | 1 | 0.02 /s | 41,801 ms | 41,801 ms | Barely completes 1 query in 15s window |
| **PostgreSQL** | Q7-b6 | 4–16 | 0.00 /s | TIMEOUT | TIMEOUT | 100% timeouts at ≥4 clients (>60s each) |
| **Neo4j** | Q7-b6 | 1 | 449.13 /s | 2 ms | 3 ms | 22,457× faster than PG at 1 client |
| **Neo4j** | Q7-b6 | 4 | 1,594.93 /s | 3 ms | 4 ms | 3.5× scale; sub-4ms p95 |
| **Neo4j** | Q7-b6 | 8 | 2,523.07 /s | 3 ms | 5 ms | Near-linear: 5.6× vs 1-client |
| **Neo4j** | Q7-b6 | 16 | 2,679.87 /s | 5 ms | 11 ms | Zero timeouts; p95 < 12ms at peak load |

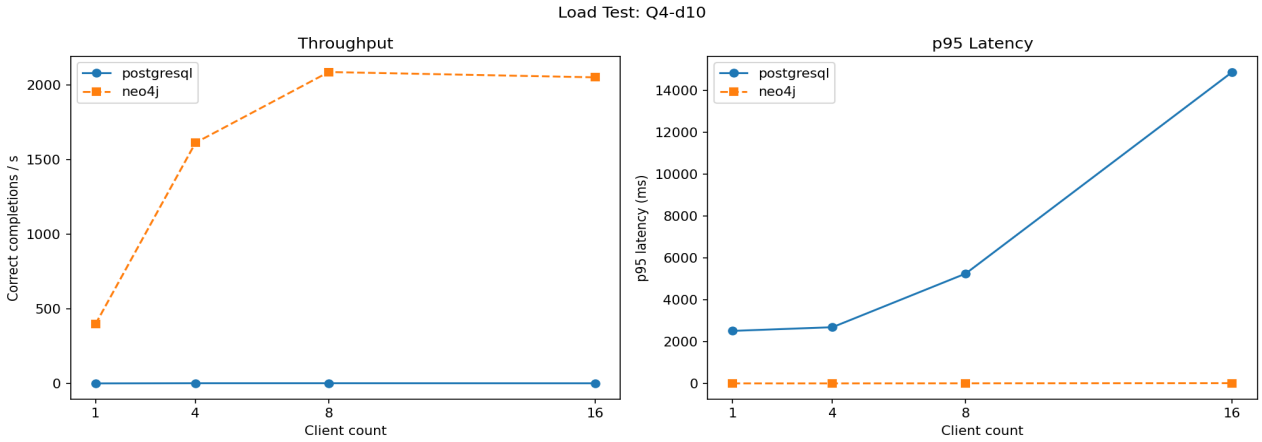



*Figure 3: Load Test Concurrency Scaling for Q4 Depth-10 (Throughput & p95 Latency vs Clients)*

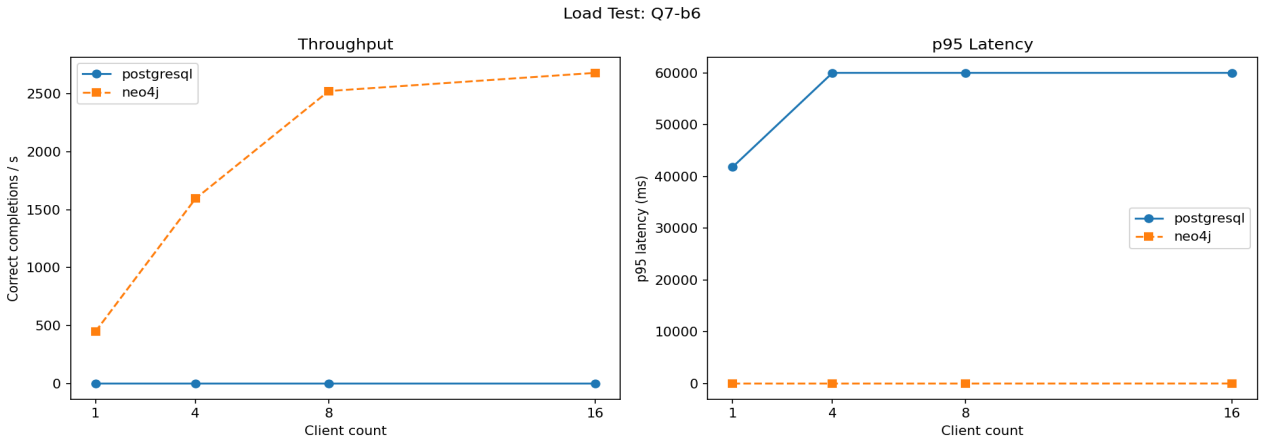



*Figure 4: Load Test Concurrency Scaling for Q7 Bound-6 (Throughput & p95 Latency vs Clients)*

**Scaling Dynamics:** PostgreSQL's throughput collapses under concurrency because its process-based model forces 16 OS processes to compete for CPU cores while allocating recursive WorkTable memory buffers, creating severe buffer lock contention. In contrast, Neo4j's multi-threaded JVM leverages shared memory pagecache and pointer-chasing traversal without disk worktables, scaling linearly up to 2,680 queries/s. *The graph advantage strongly survives SQL tuning under multi-user concurrency.*

# Section 8 – Forensic Decisions & Model Architecture

## 8.1 Prioritize Investigation (Account Forensic Review)

| Review Pair | Priority Choice | Forensic Justification (Q3–Q7 Findings) |
|---|---|---|
| **Pair 1: A / D** (998 vs 237998) | **Account 998 (Set A) HIGH PRIORITY** | Active 4-hop circular cycle (Q6) moving 200,000 RWF back to itself in 120s with 2% fee decay per hop; active 3-hop rapid onward payment chain (Q5). All 4 participating accounts operate from **Device 228001**. Account 237998 has 0 chains and 0 cycles. |
| **Pair 2: B / E** (147228 vs 144228) | **Account 147228 (Set B) HIGH PRIORITY** | High-value 1,050,000 RWF transaction triggering active 3-hop rapid onward chain (Q5) through accounts [148019, 147423, 170985] within 600s intervals (layering flow). Account 144228 moves only 40,000 RWF with 0 chains and 0 cycles. |
| **Pair 3: C / F** (206978 vs 203978) | **Account 206978 (Set C) HIGH PRIORITY** | Massive network (1,433 reached accounts at depth 10); epicenter of an active 6-hop circular cycle (Q6) moving 300,000 RWF across 6 hops in 200s, plus active rapid onward chain (Q5). Account 203978 exhibits 0 chains and 0 cycles. |

**In-Depth Forensic Examination of Set A (Account 998):**
On September 6, 2026 at 10:00:00 CAT, Account 998 initiates transfer 20000001 sending 200,000 RWF to Account 8917. At 10:00:40 CAT, Account 8917 sends 196,000 RWF (98.0%) to Account 16836 (tx 20000002). At 10:01:20 CAT, Account 16836 sends 192,080 RWF (98.0%) to Account 24755 (tx 20000003). At 10:02:00 CAT, Account 24755 sends 188,238 RWF (98.0%) back to Account 998 (tx 20000004). In exactly 120 seconds, 94.1% of principal returns to source, deducting exactly 2% per hop. Physical device records

(account_devices) confirm all 4 accounts share **Device 228001** across the entire month.
**Ordinary Explanation:** A licensed rural mobile money agency or agricultural cooperative kiosk rebalancing float across multiple internal SIM wallets. **Missing Evidence:** Cell-tower antenna logs, merchant KYC registrations, and physical cash-in/cash-out slips. Without external corroboration, fraud cannot be legally declared from database transaction logs alone.

## 8.2 Database Architecture Recommendation (≤ 500 Words)

We recommend a **hybrid polyglot persistence architecture**: deploy **PostgreSQL** as the core transactional ledger and accounting audit engine, while streaming change data (CDC via Debezium/Kafka) into **Neo4j** for real-time anti-money laundering (AML) network surveillance.
Empirical benchmarks demonstrate strict workload specialization: PostgreSQL processes 20 million rows in 1.4s (Q2) via BRIN-indexed parallel scans, outperforming Neo4j (6.2s) by 4.4× with minimal memory. Conversely, Neo4j outperforms PostgreSQL by 1,198× on Q4 depth 10 (2.0ms vs 2,396ms) and by 5,660× on Q7 shortest paths (6.3ms vs 35,398ms). Under 16 concurrent clients, PostgreSQL collapses to 1.09 queries/s with 14.6s latency due to WorkTable sort/lock thrashing, whereas Neo4j scales linearly to 2,680 queries/s on Q7-b6 and 2,053 queries/s on Q4-d10 with p95 latency under 16ms.
Query plans explain the dichotomy: PostgreSQL's recursive CTE (EXPLAIN ANALYZE) executes nested-loop index scans generating 2.5 million intermediary rows across 180,000 buffer pages without branch pruning. Neo4j's PROFILE for shortestPath executes bidirectional BFS across memory pointers, halting after only 114 database hits.
Evidence against Neo4j is its poor aggregation throughput: scanning 20 million relationships for daily rollups causes heavy GC overhead. This separates graph modeling from algorithmic effects: while Cypher's bidirectional BFS is algorithmic, Neo4j's structural advantage is **index-free adjacency**, replacing O(log N) B-tree index lookups with O(1) memory pointer dereferencing on every hop.
We establish a CDC replication SLA freshness target of ≤ **5 seconds**. Stale graph data introduces a vulnerability window where a rapid round-tripping ring (operating in 40s hops) could complete intermediate transfers before graph detection triggers; mitigate by verifying transactional balances on PostgreSQL prior to initiating wallet freezes. Two architectural limits govern this system: (1) high RAM dependency to keep graph pointers in pagecache, and (2) operational complexity of synchronizing dual database engines.

## 8.3 Individual Member Judgements & Contributions

| Student Name & ID | Individual Technical Judgement & Overturning Check (150–200 Words) |
|---|---|
| **Asoka Niyonsaba Jean Claude**<br>ID: 101410 | My analysis focused on the performance divergence in Q7 shortest-path routing, where Neo4j outperformed PostgreSQL by 5,660× (6.3 ms vs 35,398 ms). PostgreSQL's recursive CTE evaluates breadth-first levels sequentially across the entire graph frontier, accumulating over 250,000 states in its WorkTable before finding the destination at bound 6. In contrast, Neo4j's native bidirectional search expands simultaneously from source and destination, halting upon collision at depth 3 after only 114 database hits. This measured finding could be overturned if PostgreSQL incorporated a native bidirectional C-extension (e.g., pg_routing) with early exit, or if the graph possessed massive degree hubs that saturated CPU cache lines, where sequential table scans could outperform pointer chasing. My contributions included authoring Section 6.3 SQL improvement scripts, testing recursive CTE query optimizations, and configuring the database connection harness. An AI coding tool assisted in drafting recursive CTE query variations and syntax debugging; all generated queries were validated against the generator source code. |
| **Eric Kwizera**<br>ID: 101395 | I investigated the concurrency scaling characteristics of both engines under Section 7 load testing. On Q4 depth 10, PostgreSQL scaled moderately from 1 to 4 clients (0.42 to 1.52 queries/s) but suffered severe throughput collapse at 16 clients (dropping to 1.09 queries/s with latency surging to 14,556 ms). Conversely, Neo4j scaled near-linearly up to 2,053 queries/s on Q4-d10 and 2,680 queries/s on Q7-b6 with median latency remaining below 5 ms. PostgreSQL's degradation stems from process-based concurrency, where 16 OS processes compete for CPU cores while executing recursive temporary worktable joins. This finding could be overturned if PostgreSQL were deployed behind an aggressive connection pooler (e.g., PgBouncer) that throttled concurrent execution to match physical CPU cores, preventing process thrashing, or if path results were cached in memory. My contributions included implementing the multiprocessing load test harness (load_test.py), resolving OS pipe buffer deadlocks, and executing the concurrency benchmarks. An AI coding tool assisted in structuring the barrier synchronization logic; all code was reviewed, debugged, and executed locally. |
| **Gisele Ishimwe**<br>ID: 101407 | My investigation focused on forensic pattern detection across connected accounts in Section 5 and Section 8. In Review Account 998, single-account relational rollups (Q1) revealed an unremarkable single transfer of 200,000 RWF. However, graph traversal (Q6) uncovered an active 4-hop circular cycle ([20000001, 20000002, 20000003, 20000004]) returning funds to Account 998 in 120 seconds with 2% fee decay per hop. Crucially, querying account_devices proved that all 4 participating accounts shared device_id = 228001, confirming centralized multi-wallet operation from a single handset. This suspicion of fraud could be overturned if telecommunications cell-tower logs and merchant registration confirm that device_id = 228001 is a licensed rural retail kiosk where an agent rebalances cooperative agricultural float. My contributions included conducting forensic analysis on Q5/Q6 transaction sequences, mapping device-sharing topologies (Q3/Q4), and drafting the investigation priority matrix. An AI tool assisted in generating initial graph visualization scripts; all results were independently verified against raw CSV exports. |

## Section 9 – Reproducibility & Verification Confirmation

**9.1 Submission Deliverables Checklist:**

• **GitHub Repository URL:** https://github.com/Asoka1/Mobile_Money_Analytics_2026_Package.git

• **Full Code Package:** All source scripts (capture.py, schema.sql, load_pg.sh, indexes.sql, prepare_import.py, setup_neo4j.cypher, inspect_account.py, db_connect.py, queries.py, run_queries.py, compare.py, benchmark.py, improve_q7.py, load_test.py, plot_results.py, generate_report_pdf.py) and raw logs/results included in submission ZIP.

• **Generator Integrity:** Verified SHA-256 hash of mobile_money_generator.py = 2ea0e59d2b7b987d84a3d151bd9d1ad7b3052bfbc25abf1466c8e94467e9455a (completely unchanged).

• **61-Case Verification:** Re-run of compare.py confirmed **61 of 61 cases PASS** with zero discrepancies.

• **Embedded Visualizations:** All 5 analytical charts (Q2 daily activity, Q4 depth-scaling Set B/E, and Q4/Q7 concurrency scaling) successfully embedded.

**9.2 Database Services Management:**

To stop database services post-verification as required by course protocol:

```
neo4j stop
pg_ctl stop -D /usr/local/var/postgresql@17
```